A tour of the ML-KEM Braid correctness proof in SM

Why the proof needs states and runs, not only `send` and `recv`

SpqrBridge

November 2026

The proof shown: `MLKEMBraid.correctness_error_le`. Last slide: its transport to `SPQR.Chunked`.

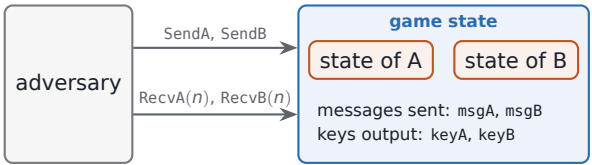
What correctness means

- ML-KEM Braid is an SCKA scheme. It is correct if it satisfies SCKA correctness.
- Two parties, A and B, call `send` and `recv` to exchange messages. Some of these calls output a key (e, k) for an epoch e.
- The network may drop, delay, reorder or replay messages. Both parties are honest.
- SCKA correctness:
  - every message that the other party sent is accepted;
  - A and B output the same key for each epoch;
  - each party outputs one key per epoch, for epochs $1, 2, 3, \ldots$ in order.
- ML-KEM Braid satisfies it except with probability $q\varepsilon$: q sends, each with a KEM decapsulation failure probability ε.

Docs: SCKA protocol correctness (the definition)

The correctness game

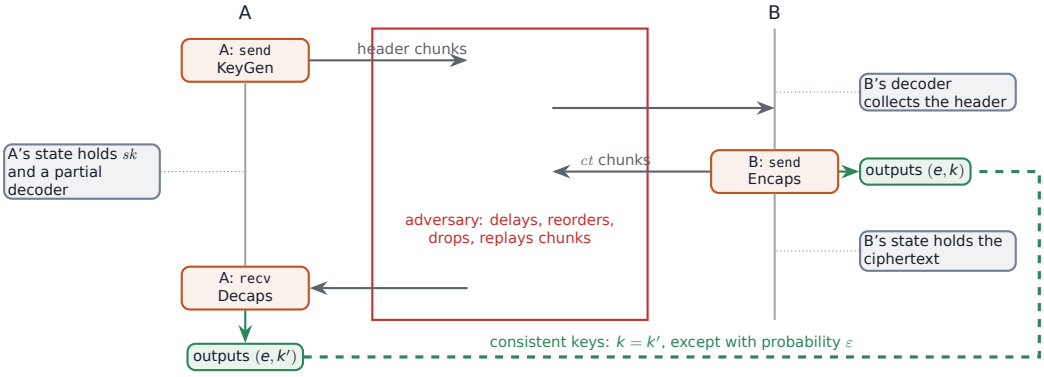
- SM states correctness as a game, `correctnessExp`. The adversary plays the network. The game returns the flag `correct`.



- `MLKEMBraid.correctness_error_le`: with at most q sends, $1 - \Pr[\texttt{correct}] \leq q\varepsilon$.

Lean: `correctnessExp`, `MLKEMBraid.correctness_error_le`. Docs: ML-KEM Braid correctness

ML-KEM Braid correctness is a property of a run

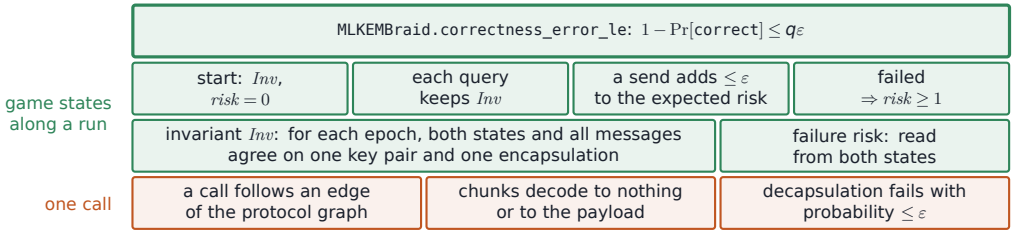


One epoch e, with A as the key generator.

- k and k' come from calls of different parties, with any number of queries between them.
- Only the party states carry sk and the ciphertext between them. So the proof needs a fact about both states that every query keeps: an invariant.

The bricks of the proof

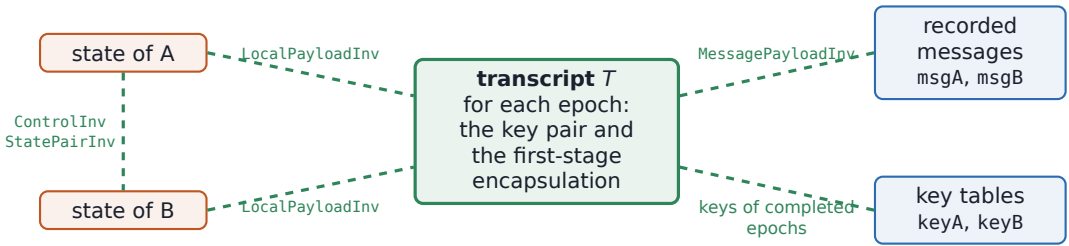
- Each row rests on the row below it.



- Only the bottom row is about single calls. Every row above it is about game states along a run.

The invariant ties both states to one transcript

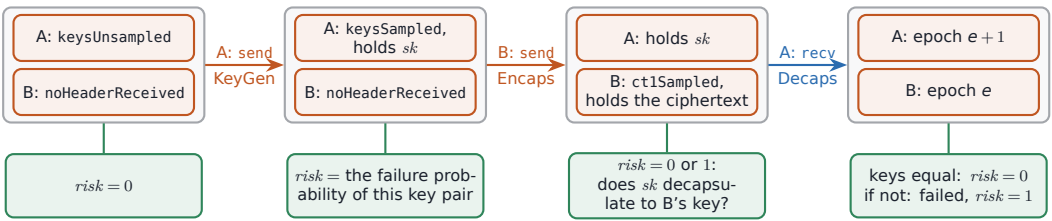
```
CorrectnessInv s := s.correct = false ∨ ∃ T, TranscriptConsistent T s
```



- T exists only in the proof. It records what the run sampled.
- Both key tables are read from the same T, so the keys are equal.
- A fact about one call cannot say “this ciphertext was made for the key pair in the other party’s state”. The invariant says it, through T.

The failure risk reads both states

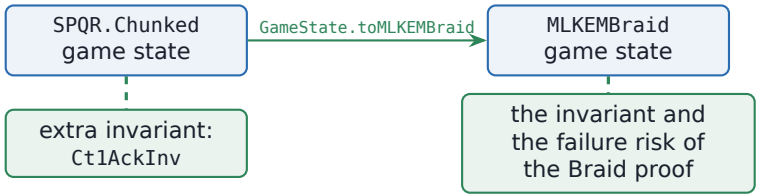
- $risk$: the probability of failure that the states already carry. One epoch e, A as the key generator.



- $risk$ reads both party states. No single call has sk and the ciphertext together.
- A send raises $\mathbb{E}[risk]$ by at most ε, and a failed game has $risk \geq 1$. So $\Pr[\mathsf{fail}] \leq \mathbb{E}[risk] \leq q\varepsilon$.

From ML-KEM Braid to `SPQR.Chunked`

- secure-messaging #398 proves the same bound for `SPQR.Chunked`, without repeating the Braid proof.



- Most queries are the Braid query on the translated state, so the Braid lemmas apply.
- One query differs: a plain vector chunk delivered to a party in `ct1Acknowledged`. The translation relabels it as an acknowledging chunk.
- #398 is not merged. The bridge pins SM at `c16e287`, which has it.

Lean: `SPQR.correctness_error_le`. Docs: SPQR correctness